CHRIST (Deemed to be University), Bangalore – 560 029

Department of Computer Science

END-TRIMESTER EXAMINATION MARCH- 2026

PG III Trimester

Programme Name: MSAIM Max. Marks: 100 Course Name: - Deep Learning Time: 3 Hrs
Course Code: MAI417-3

General Instructions

- All rough work should be done in the answer script. Do not write or scribble on the question paper except your registration number.
- Verify the Course code / Course title & number of pages of questions in the question paper.
- Ensure your mobile phone is switched off and placed at the designated place in the hall. ●

Malpractices will be viewed very seriously.

Course Outcomes (COs): The students will able to
CO1: Implement deep feedforward and backpropagation networks for classification. CO2: Analyze regularization techniques for optimizing deep learning models.
CO3: Apply convolutional and recurrent neural networks for vision and sequence processing tasks. CO4: Evaluate autoencoders for feature extraction and data compression.
CO5: Compare different deep learning architectures for performance improvement.

End Trimester Examination – Hackathon

NeuralHack 2026 invites you to apply the power of **Deep Learning** to solve meaningful real world problems. This hackathon based examination challenges you to design and implement an intelligent system using techniques learned throughout the course.

Your task is to build an **end to end deep learning solution** using the dataset provided during the examination. The solution should demonstrate your ability to understand a problem, model it mathematically, design an appropriate deep learning architecture, and develop a working prototype.

You may use frameworks such as **TensorFlow, Keras, or PyTorch**.

This challenge evaluates your **technical understanding, creativity, and ability to translate deep learning concepts into practical applications**.

Hackathon Challenge

Design and implement an **end to end deep learning model for a real world data driven problem** using

the appropriate dataset for your problem statement during the examination.

Your solution should demonstrate any of the application studied in the course such as

Deep Feedforward Networks
Regularization Techniques
Convolutional Neural Networks
Recurrent Neural Networks or LSTM
Autoencoder Architectures

Your model should be able to learn from the data and produce meaningful predictions or representations.

Tasks

Problem Definition (CO1, L3) Clearly define the learning task by identifying

• input representation
• expected output
• type of learning task such as classification, prediction, sequence modeling, or representation learning

Explain the real world relevance of the problem.

Mathematical Modeling (CO2, L4) Provide a concise theoretical explanation including

• formulation of the selected deep learning architecture
• loss function used for optimization
• training strategy and parameter learning approach
• evaluation metrics used to measure model performance

Model Design and Implementation (CO3,L5)

Design and implement an appropriate deep learning architecture using techniques covered in the course.Regularization methods such as dropout, L1, or L2 regularization may be incorporated where necessary.Implementation can be carried out using **TensorFlow, Keras, or PyTorch**.

Model Training and Evaluation (CO4,L5)
Your implementation should include

• data preprocessing and preparation
• model construction and training
• validation and performance evaluation

• analysis and interpretation of model outputs

User Interface Demonstration (CO5,L6)

To demonstrate the usability of your system, develop a **simple user interface that allows interaction with the trained model**.

The interface should enable a user to provide an input sample and observe the model prediction or output.The interface may be implemented using

• a basic web interface
• a notebook based interactive interface
• a lightweight application interface

Expected Deliverables

Students must submit

• Working deep learning implementation using TensorFlow, Keras, or PyTorch
• Output predictions or results generated by the model
• A short written explanation describing the mathematical formulation and architecture design
• A simple user interface demonstrating the functionality of the developed model

Suggested Inspiration Scenarios

The following scenarios are **only examples to spark ideas**. You are **not required to restrict your solution to these problems**.

1. Smart Agriculture Monitoring
   Detect crop diseases or analyze plant health from image data.
2. Disaster Information Analysis
   Identify emergency related messages from social media or communication data.
3. Urban Traffic Intelligence
   Analyze traffic patterns to estimate congestion or predict traffic flow.
4. Healthcare Monitoring
   Detect abnormal patterns in patient data or analyze medical images.
5. Environmental Monitoring
   Predict pollution levels or environmental changes from sensor data.

Open Your Mind to More Possibilities

The examples above are only a starting point. NeuralHack encourages you to explore broader domains such as

1. Urban Development
2. Healthcare and Wellbeing
3. Education and Accessibility
4. Agriculture and Food Security
5. Disaster Management and Public Safety

Use the knowledge gained in this course to **build an intelligent system that demonstrates deep learning in action**.

Think creatively.
Design intelligently.
Build something meaningful.

Evaluation Rubrics :

Evaluation Criteria	**Description**	**Marks**
Problem Formulation and Understanding	Clarity in defining the real world problem, identification of input and output, and explanation of the learning objective	10
Mathematical Modeling	Clear explanation of the deep learning formulation including architecture description, loss function, optimization strategy, and evaluation metrics	20
Model Architecture Design	Appropriateness of the chosen deep learning architecture and correct application of concepts such as CNN, RNN, feedforward networks, autoencoders, and regularization	20
Implementation and Code Quality	Correctness of implementation, logical structure of code, use of appropriate frameworks such as TensorFlow, Keras, or PyTorch, and reproducibility	15
Data Preparation and Training Process	Data preprocessing, dataset handling, model training, and appropriate validation strategy	10
Model Performance and Evaluation	Model accuracy or performance, interpretation of results, and ability to analyze outcomes	10
User Interface Demonstration	Development of a simple interface to interact with the model and demonstrate predictions or outputs	10

Innovation and Real World Relevance	Creativity of the solution and its potential societal or practical impact	5

Revised Bloom's Taxonomy (RBT) Levels :		
L1 – Remembering	L2 – Understanding	L3 – Applying
L4 – Analyzing	L5 – Evaluating	L6 - Creating

Problem Statement:

The objective of this project is to develop a deep learning-based system for the automated detection of pneumonia from chest X-ray images. Pneumonia is a serious lung infection that can be life-threatening if not diagnosed early. Manual diagnosis by radiologists can be time-consuming and may be prone to human error, especially in high-volume healthcare settings.

In this work, we aim to build a Convolutional Neural Network (CNN) model using transfer learning to classify chest X-ray images into two categories: Normal and Pneumonia. The model takes a preprocessed X-ray image as input and outputs the probability of the presence of pneumonia.

This system can assist healthcare professionals by providing quick and accurate preliminary screening, thereby improving diagnostic efficiency and supporting early treatment decisions, particularly in resource-limited environments.

Approach:

The proposed approach involves using a Convolutional Neural Network (CNN) with transfer learning to classify chest X-ray images. The input images are first preprocessed through resizing and normalization, along with data augmentation to improve generalization. A pretrained MobileNetV2 model is used as a feature extractor, followed by fully connected layers for classification. The model is trained using binary crossentropy loss and optimized with the Adam optimizer. Regularization techniques such as dropout and data augmentation are applied to reduce overfitting. Finally, the model is evaluated using accuracy, confusion matrix, and classification metrics.

CODE:

```
import numpy as np
import matplotlib.pyplot as plt
import os
import seaborn as sns

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint

from sklearn.metrics import classification_report, confusion_matrix
```

```
base_path = "/content/drive/MyDrive/chest_xray"
```

```
train_dir = base_path + "/train"
val_dir   = base_path + "/val"
test_dir  = base_path + "/test"
```

```
import os

print(os.listdir(train_dir))
```

```
['NORMAL', 'PNEUMONIA']
```

```python
IMG_SIZE = 224
BATCH_SIZE = 16

train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=15,
    zoom_range=0.2,
    horizontal_flip=True
)

val_test_datagen = ImageDataGenerator(rescale=1./255)

train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

val_generator = val_test_datagen.flow_from_directory(
    val_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary'
)

test_generator = val_test_datagen.flow_from_directory(
    test_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='binary',
    shuffle=False
)
```

```
Found 5216 images belonging to 2 classes.
Found 16 images belonging to 2 classes.
Found 624 images belonging to 2 classes.
```

```python
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(IMG_SIZE, IMG_SIZE, 3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = False
```

```python
from tensorflow.keras import regularizers
```

```python
model = models.Sequential([
    base_model,

    layers.GlobalAveragePooling2D(),
    layers.BatchNormalization(),
    layers.Dense(128, activation='relu',kernel_regularizer=tf.keras.regularizers.l2(0.001)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(1, activation='sigmoid')
])
```

```python
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.0001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```python
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

checkpoint = ModelCheckpoint(
    "best_model.h5",
    monitor='val_accuracy',
    save_best_only=True
)
```

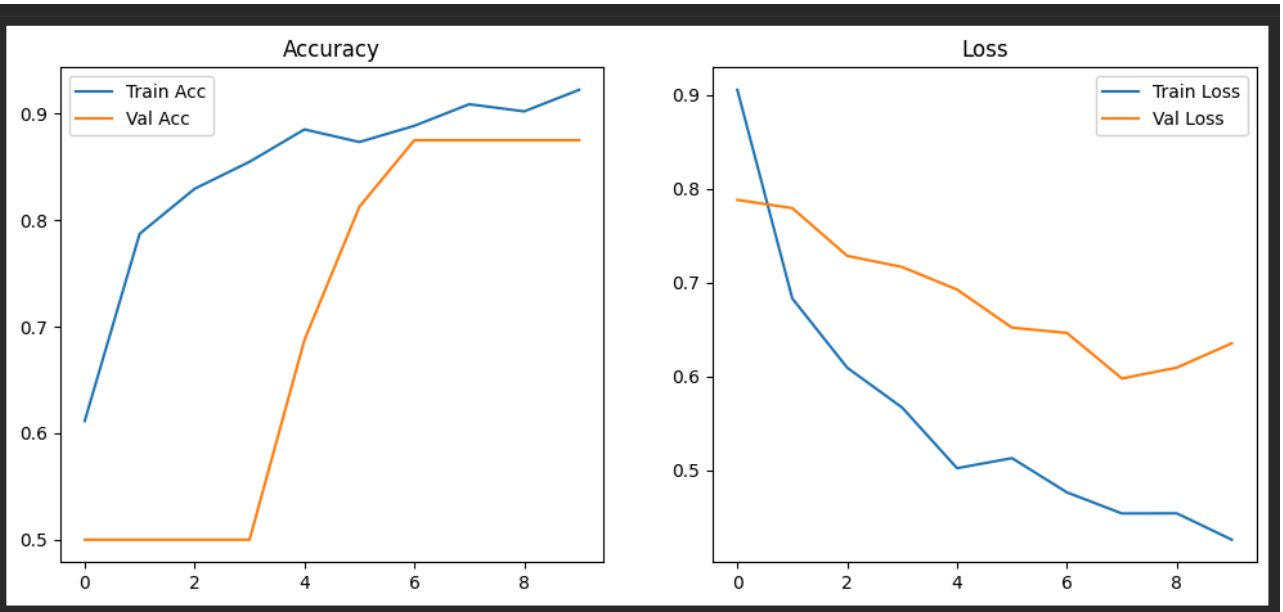
```python
steps_per_epoch = 600 // BATCH_SIZE
validation_steps = 200 // BATCH_SIZE
```

```python
"""history = model.fit(
    train_generator,
    validation_data=val_generator,
    epochs=10,
    callbacks=[early_stop, checkpoint]
)"""
history = model.fit(
    train_generator,
    steps_per_epoch=steps_per_epoch,
    validation_data=val_generator,
    validation_steps=validation_steps,
    epochs=10
)
```

```
Epoch 1/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 116s 3s/step - accuracy: 0.6115 - loss: 0.9051 - val_accuracy: 0.5000 - val_loss: 0.7878
Epoch 2/10
 1/37 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.8125 - loss: 0.7518/usr/local/lib/python3.12/dist-packages/keras/
  self._interrupted_warning()
37/37 ━━━━━━━━━━━━━━━━━━━━ 83s 2s/step - accuracy: 0.7872 - loss: 0.6830 - val_accuracy: 0.5000 - val_loss: 0.7792
Epoch 3/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 79s 2s/step - accuracy: 0.8294 - loss: 0.6092 - val_accuracy: 0.5000 - val_loss: 0.7283
Epoch 4/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 69s 2s/step - accuracy: 0.8547 - loss: 0.5668 - val_accuracy: 0.5000 - val_loss: 0.7164
Epoch 5/10
```

```
Epoch 5/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 54s 1s/step - accuracy: 0.8851 - loss: 0.5022 - val_accuracy: 0.6875 - val_loss: 0.6924
Epoch 6/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 57s 2s/step - accuracy: 0.8733 - loss: 0.5129 - val_accuracy: 0.8125 - val_loss: 0.6519
Epoch 7/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 41s 1s/step - accuracy: 0.8885 - loss: 0.4764 - val_accuracy: 0.8750 - val_loss: 0.6463
Epoch 8/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 46s 1s/step - accuracy: 0.9088 - loss: 0.4540 - val_accuracy: 0.8750 - val_loss: 0.5977
Epoch 9/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 35s 923ms/step - accuracy: 0.9021 - loss: 0.4541 - val_accuracy: 0.8750 - val_loss: 0.6093
Epoch 10/10
37/37 ━━━━━━━━━━━━━━━━━━━━ 92s 1s/step - accuracy: 0.9223 - loss: 0.4261 - val_accuracy: 0.8750 - val_loss: 0.6352
```

```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.legend()
plt.title("Accuracy")

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.legend()
plt.title("Loss")

plt.show()
```



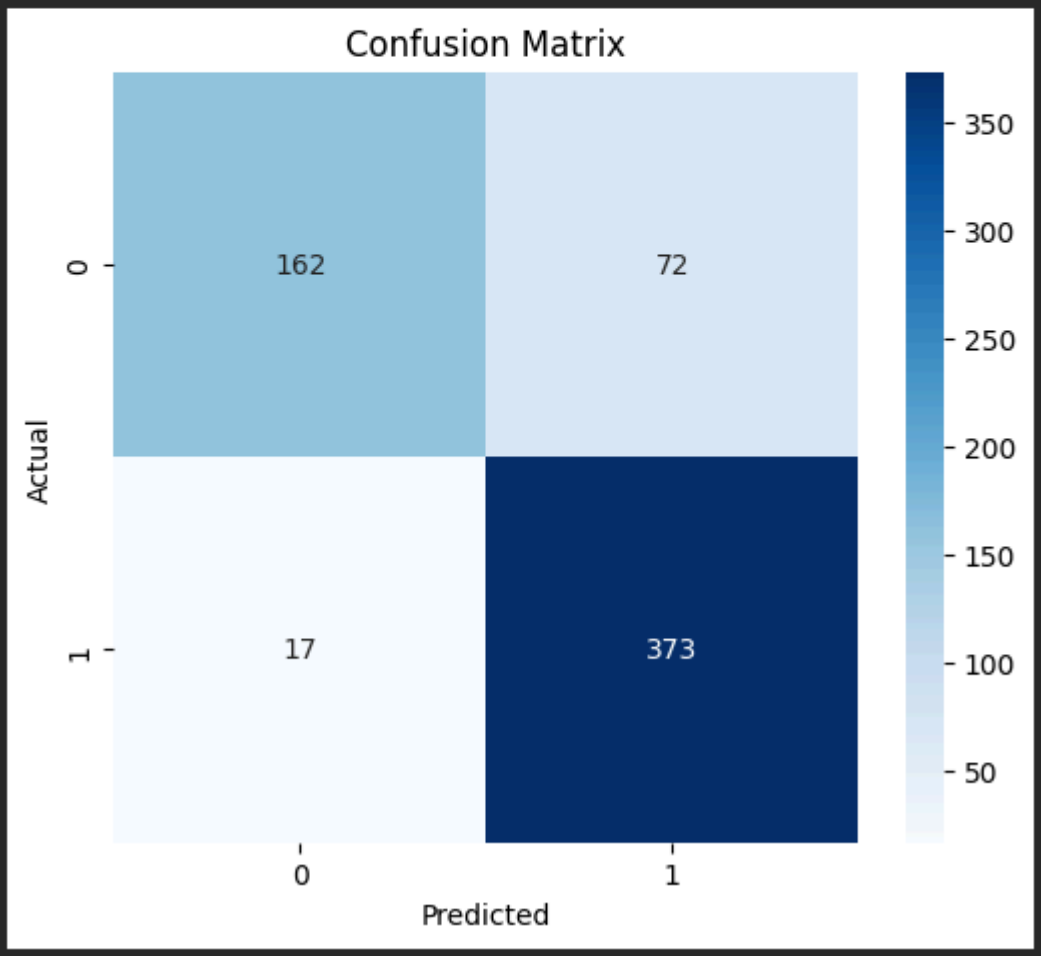

```
test_loss, test_acc = model.evaluate(test_generator)
print("Test Accuracy:", test_acc)
```

```
39/39 ━━━━━━━━━━━━━━━━━━━━ 6s 154ms/step - accuracy: 0.8574 - loss: 0.5364
Test Accuracy: 0.8573718070983887
```

```
predictions = model.predict(test_generator)
y_pred = (predictions > 0.5).astype(int)

y_true = test_generator.classes
```

```
39/39 ━━━━━━━━━━━━━━━━━━━━ 11s 165ms/step
```

```python
y_true = test_generator.classes
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

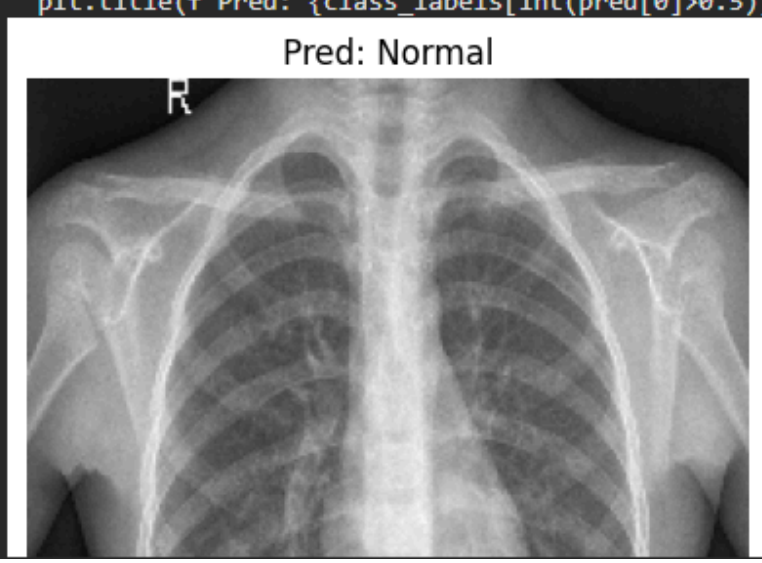


```python
print(classification_report(y_true, y_pred, target_names=['Normal','Pneumonia']))
```

```
              precision    recall  f1-score   support

      Normal       0.91      0.69      0.78       234
   Pneumonia       0.84      0.96      0.89       390

    accuracy                           0.86       624
   macro avg       0.87      0.82      0.84       624
weighted avg       0.86      0.86      0.85       624
```

```python
class_labels = ['Normal', 'Pneumonia']

for i in range(5):
    img, label = test_generator[i]
    pred = model.predict(img)

    plt.imshow(img[0])
    plt.title(f"Pred: {class_labels[int(pred[0]>0.5)]}")
    plt.axis('off')
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 6s 6s/step
/tmp/ipykernel_1274/2810938840.py:8: DeprecationWarning: Conversion of an array wi
  plt.title(f"Pred: {class_labels[int(pred[0]>0.5)]}")
```



```
model.save("/content/drive/MyDrive/pneumonia_model.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `
```

```
!pip install gradio
```

```
import gradio as gr
import tensorflow as tf
import numpy as np
from PIL import Image
```

```
model = tf.keras.models.load_model(
    "/content/drive/MyDrive/pneumonia_model.h5"
)
```

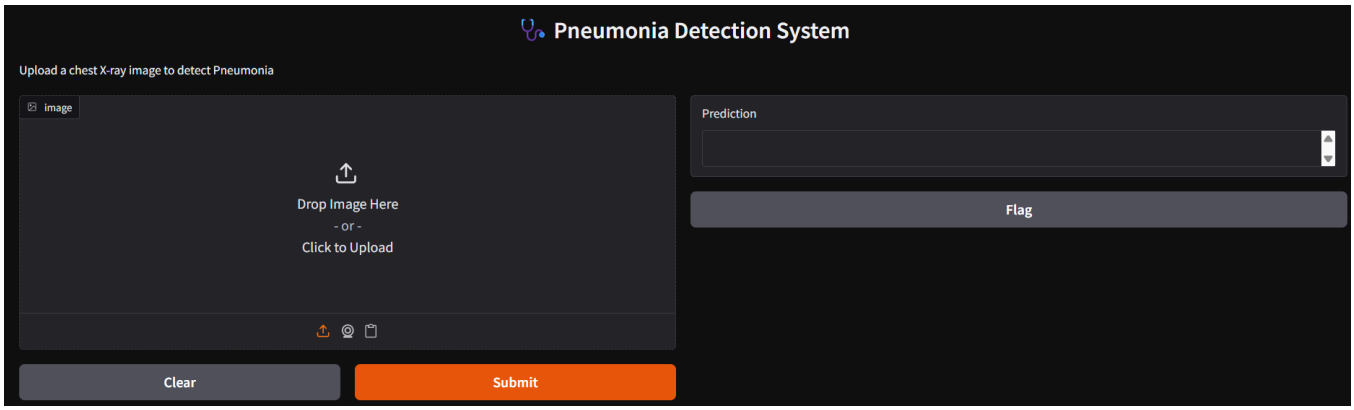
```
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to
```

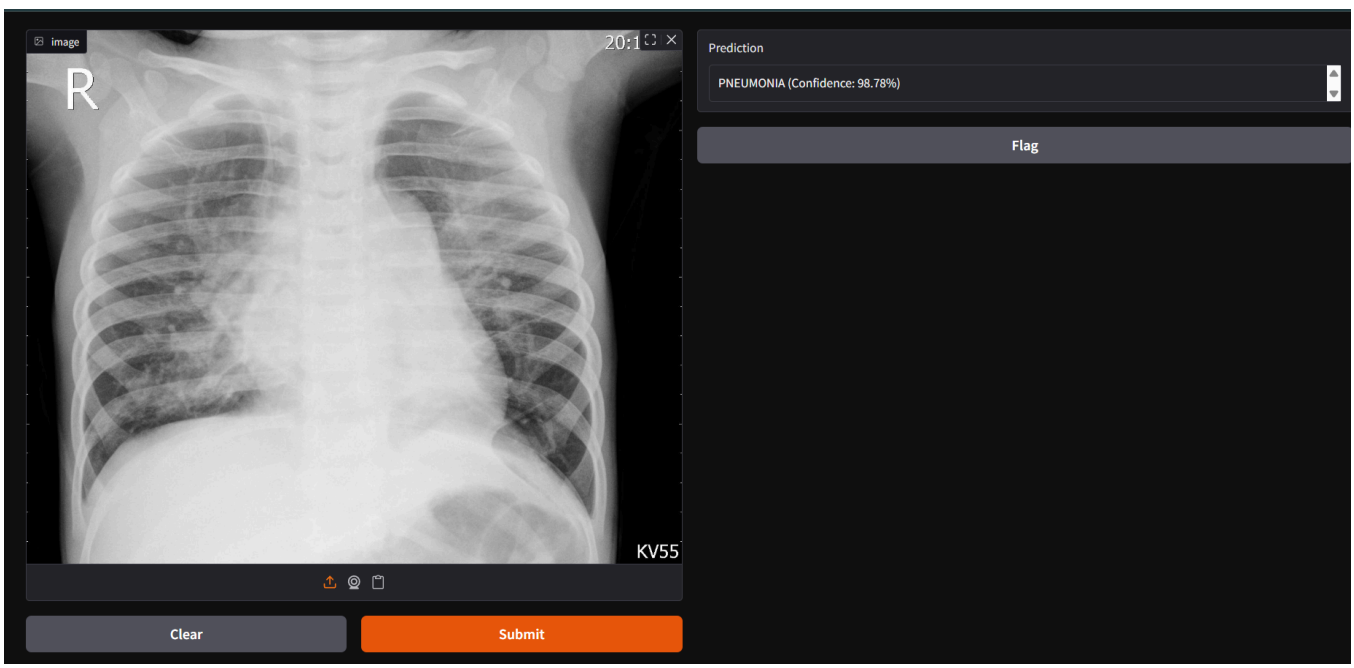
```
IMG_SIZE = 224

def predict(image):
    # Resize
    img = image.resize((IMG_SIZE, IMG_SIZE))

    # Convert to array
    img = np.array(img) / 255.0

    # Expand dims
    img = np.expand_dims(img, axis=0)

    # Predict
    pred = model.predict(img)[0][0]

    if pred > 0.5:
        label = "PNEUMONIA"
        confidence = float(pred)
    else:
        label = "NORMAL"
        confidence = float(1 - pred)

    return f"{label} (Confidence: {confidence*100:.2f}%)"
```

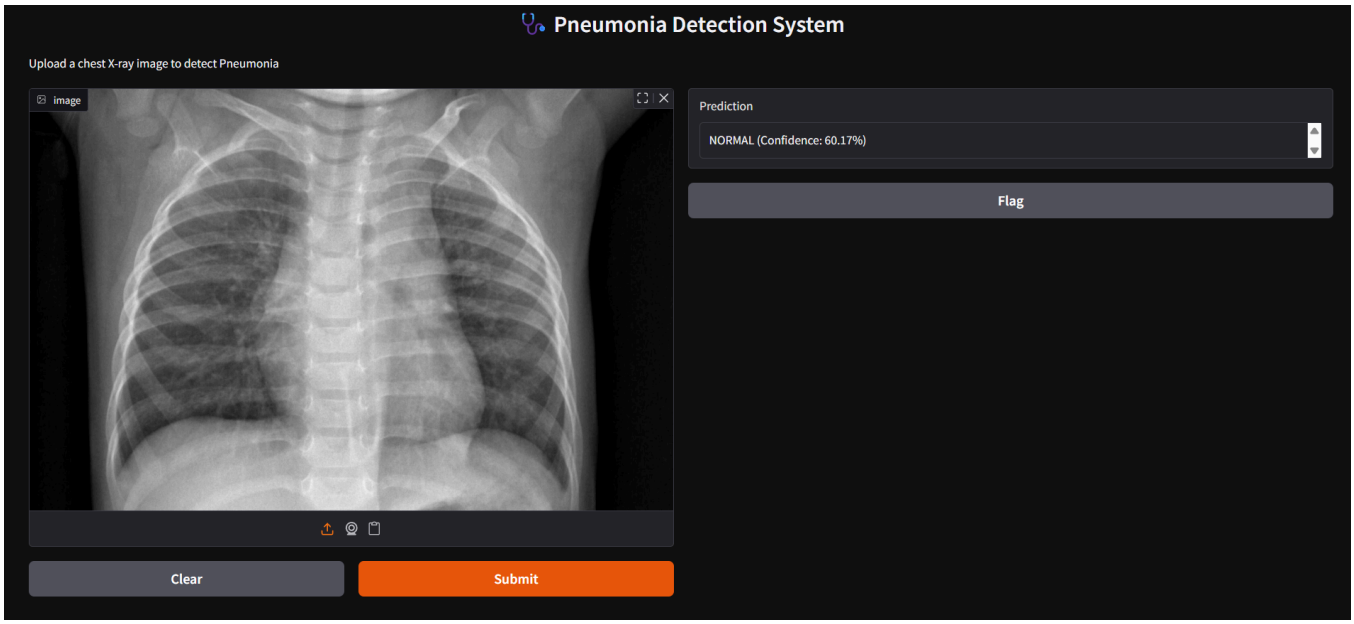
```python
interface = gr.Interface(
    fn=predict,
    inputs=gr.Image(type="pil"),
    outputs=gr.Textbox(label="Prediction"),
    title="🩺 Pneumonia Detection System",
    description="Upload a chest X-ray image to detect Pneumonia"
)
```

```python
interface.launch()
```

```
It looks like you are running Gradio on a hosted Jupyter notebook, which requires `share=True`.

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: https://8ee970dc9e6ffc2098.gradio.live

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio dep
```






Pneumonia Detection System
Upload a chest X-ray image to detect Pneumonia
image
Prediction
NORMAL (Confidence: 60.17%)
Flag
Clear
Submit